Lesson 8 Teacher's Key: Under Pressure

Godot Engine - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

A second, independent signal (escaped) added alongside an existing one (destroyed) - the cleanest possible illustration in the course of two distinct event types living side by side without interfering, extending the exact pattern that made Godot immune to the Lesson 7 bug.

Level 1 - Full Worked Solution

- signal escaped is declared in enemy.tscn's script alongside the pre-existing signal destroyed - same mechanism, different name, different meaning.
- escaped.emit() is added to the EXISTING escape branch in _process(), right before the pre-existing queue_free() call - one new line in an already-correct location.
- lives and on_enemy_escaped() are added to GameManager's script, alongside its existing enemies_remaining and on_enemy_destroyed() - same node, same pattern, different signal.
- The main scene connects BOTH signals per spawned enemy, both targeting GameManager (destroyed to $GameManager.on_enemy_destroyed from Lesson 6, escaped to $GameManager.on_enemy_escaped new this week) - two independent connections, two independent handler functions, one dedicated manager node.

Highlighted lines are new this week; everything else is code you already have.

```
# enemy.tscn's script:
signal escaped

func _process(delta):
    position.y += enemy_speed * delta
    if position.y > 700:
        escaped.emit()
        queue_free()

# GameManager's script - add alongside enemies_remaining and
on_enemy_destroyed():
var lives := 3

func on_enemy_escaped():
    lives -= 1
    if lives <= 0:
        get_node("../LoseText").show()
```

```
# Main scene's script - in the spawn loop, alongside connecting destroyed:
new_enemy.escaped.connect($GameManager.on_enemy_escaped)
```

Why This Structure

- Godot's signal system makes this lesson's core idea almost visually obvious: two separate signals, two separate handlers, two separate counters, all owned by the same GameManager node - there's no shared hook to misuse the way GameMaker's Destroy event or 001GC's Defeated trigger could be misused.
- This is worth explicitly naming as the payoff for Godot's Lesson 6/7 structure: because GameManager already owned all counting, and escape-cleanup and hit-detection were already fully separate code paths, adding a second signal and a second handler required zero changes to either existing one.

Level 2 / Level 3 Talking Points

- Level 3 sound, worked (Godot): GameManager gets two AudioStreamPlayer children, LifeLostSound (sfx_lifelost.wav) and GameOverSound (sfx_gameover.wav). In on_enemy_escaped(): lives -= 1, then if lives <= 0: get_node("../LoseText").show() and $GameOverSound.play(), else: $LifeLostSound.play(). Plain AudioStreamPlayer, not 2D, since this is UI-level audio.
- Ask students what would happen if _on_hitbox_area_entered() emitted BOTH destroyed and escaped by mistake (a copy-paste error) - a good diagnostic exercise, since the symptom (shooting costs a life) is confusing without understanding the two-signal structure.
- Level 3 structure: point out that a project could keep adding new signals this way indefinitely (poisoned, healed, upgraded, etc.), all still handled on GameManager, without ever needing to touch existing signal-handling code - this is a preview of how larger, real projects stay maintainable.

Level 2 - complete code:

```
# Level 2 is verification only - the Level 1 code is already correct, so
no lines change.
# Checks: shooting an enemy never changes lives (only the kill path
changes enemies_remaining);
# an escape never changes enemies_remaining (only the escape path changes
lives);
# "Game Over" appears once, when lives first reaches 0.
```

Level 3 - complete code (all three goals combined):

Highlighted lines are the Level 3 additions on top of the Level 1 code.

```
# GameManager's script - Level 3 (children: LifeLostSound and
GameOverSound, both plain AudioStreamPlayer;
# a Label named LivesText next to LoseText in the main scene):
var lives := 3

func on_enemy_escaped():
    lives -= 1
    get_node("../LivesText").text = "Lives: " + str(lives)
    if lives <= 0:
        get_node("../LoseText").show()
        $GameOverSound.play()
```

```
    else:
        $LifeLostSound.play()
```

Common Student Mistakes

- Emitting escaped from _on_hitbox_area_entered() by mistake, or destroyed from _process()'s escape branch by mistake - either one causes cross-contamination between lives and enemies_remaining.
- Writing on_enemy_escaped() on the main scene's own script instead of GameManager's - even if the logic is otherwise correct, this splits counting logic across two different owners instead of keeping it on the one dedicated node.
- Forgetting to connect the new escaped signal in the spawn loop (only destroyed connected, copied from Lesson 6 without adding the new line) - lives never changes even though the signal fires correctly.
- LoseText and WinText positioned at the same screen coordinates, causing them to overlap if both happen to show near the same time.